

JavaFX

Mickaël Foursov¹

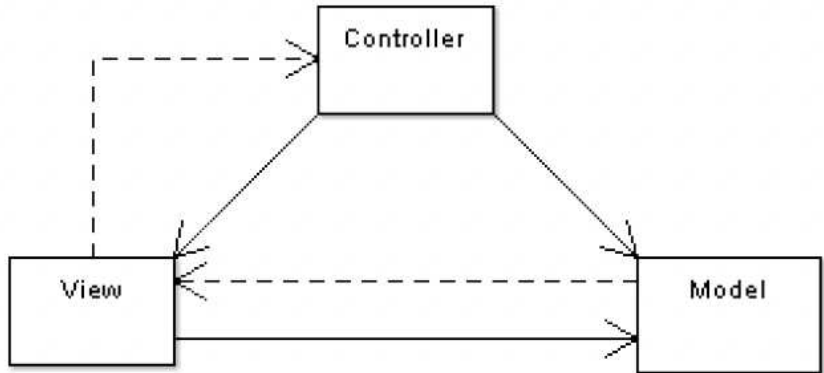
¹ISTIC, Université de Rennes

Licence Miage, février 2024

- JavaFX est une bibliothèque Java pour faciliter la création des applications lourdes, créée pour remplacer Swing.
- Elle permet de créer des IHM en XML (FXML pour être précis).
Il est possible de concevoir l'IHM sans écrire de code, avec l'outil Wysiwyg *JavaFX Scene Builder*.
- Les feuilles de style CSS permettent de séparer l'apparence et le style de l'implémentation.
- Ainsi le développeur peut se concentrer sur le codage.
- JavaFX est inclus dans la version standard de Java à partir de la version 8.

- Sun Microsystems annonce la création JavaFX en mai 2007.
- JavaFX 1.0 est sortie en décembre 2008.
- JavaFX 2.0 est sortie en octobre 2011.
- La dernière version s'appelle JavaFX 8.
- Elle fait partie maintenant de la distribution standard de Java.

Patron de conception MVC (Model–View–Controller)



Source : Zigurd Medniks, Laird Dornin, G. Blake Meike & Masumi Nakamura
“Programming Android”, O’Reilly, 2012.

Patron de conception MVC (Model–View–Controller)

- Le modèle est le coeur de l'application. Le modèle est portée sur les données.
- La vue est la visualisation du modèle ou, plus généralement, la partie de l'application en charge de l'affichage, du son, etc.
- Le contrôleur est la partie de l'appli qui répond aux actions extérieurs : un click sur un clavier, un mouvement de la souris, etc. C'est implémenté avec une file d'évènements.

La mise en page définit la structure visuelle d'une IHM. Il est possible de déclarer la mise en page dans un fichier XML ou les instancier au moment de l'exécution.

Déclarer l'IHM en XML permet de :

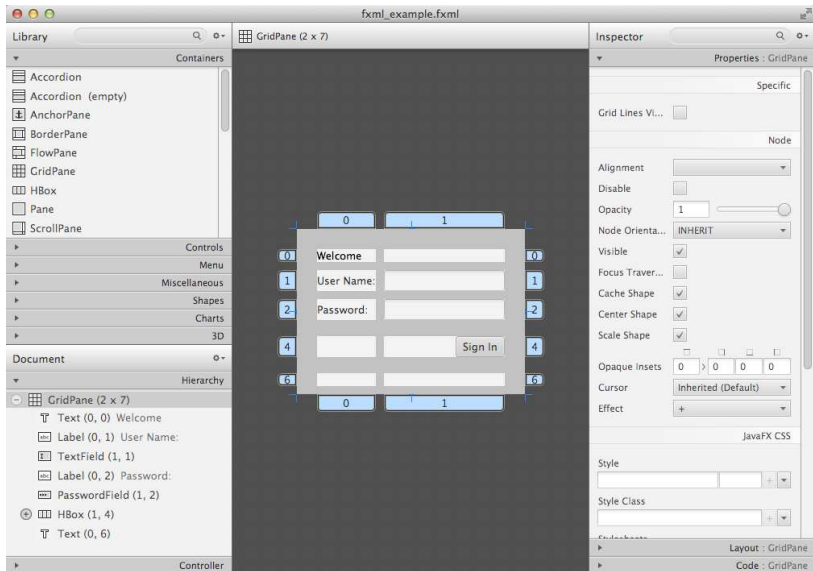
- mieux séparer la présentation de l'appli du code qui contrôle son comportement
- modifier l'IHM dans modifier le code
- visualiser l'IHM et déboguer plus facilement

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.geometry.Insets?>
4 <?import javafx.scene.control.*?>
5 <?import javafx.scene.layout.*?>
6 <?import javafx.scene.text.Text?>
7
8 <GridPane xmlns="http://javafx.com/javafx/8" xmlns:fx="http://javafx.com/fxml/1"
9     fx:controller="controller.FirstController"
10     alignment="center" hgap="10" vgap="10">
11     <padding>
12         <Insets top="25" right="25" bottom="10" left="25"/>
13     </padding>
14     <Text text="Welcome"
15         GridPane.columnIndex="0" GridPane.rowIndex="0" GridPane.columnSpan="2"/>
16     <Label text="User Name:"
17         GridPane.columnIndex="0" GridPane.rowIndex="1"/>
18     <TextField
19         GridPane.columnIndex="1" GridPane.rowIndex="1"/>
20     <Label text="Password:"
21         GridPane.columnIndex="0" GridPane.rowIndex="2"/>
22     <PasswordField fx:id="passwordField"
23         GridPane.columnIndex="1" GridPane.rowIndex="2"/>
24     <HBox spacing="10" alignment="bottom_right"
25         GridPane.columnIndex="1" GridPane.rowIndex="4">
26         <Button text="Sign In" onAction="#handleButtonAction"/>
27     </HBox>
28     <Text fx:id="actiontarget"
29         GridPane.columnIndex="0" GridPane.columnSpan="2"
30         GridPane.halignment="RIGHT" GridPane.rowIndex="6"/>
31 </GridPane>

```

Vue : fxml_example.fxml



Contrôleur et Main

```
1 public class Main extends Application {
2     @Override
3     public void start(Stage stage) throws Exception {
4         Parent root = FXMLLoader.load(getClass()
5             .getResource("view/fxml_example.fxml"));
6         Scene scene = new Scene(root, 300, 275);
7         stage.setTitle("FXML Welcome");
8         stage.setScene(scene);
9         stage.show();
10    }
11    public static void main(String[] args) {
12        Application.launch(args);
13    }
14 }
```

```
1 public class FirstController {
2     @FXML
3     private Text actiontarget;
4     @FXML
5     protected void handleButtonAction(ActionEvent event) {
6         actiontarget.setText("Sign in button pressed");
7     }
8 }
```

```
1 .root {
2     -fx-background-image: url("background.jpg");
3 }
4 #welcome-text {
5     -fx-font-size: 32px;
6     -fx-font-family: "Arial Black";
7     -fx-fill: #818181;
8     -fx-effect: innershadow(three-pass-box, rgba(0,0,0,0.7), 6, 0.0, 0, 2);
9 }
10 .label {
11     -fx-font-size: 12px;
12     -fx-font-weight: bold;
13     -fx-text-fill: #333333;
14     -fx-effect: dropshadow(gaussian, rgba(255,255,255,0.5), 0, 0, 0, 1);
15 }
16 .button {
17     -fx-text-fill: white;
18     -fx-font-family: "Arial Narrow";
19     -fx-font-weight: bold;
20     -fx-background-color: linear-gradient(#61a2b1, #2A5058);
21     -fx-effect: dropshadow(three-pass-box, rgba(0,0,0,0.6), 5, 0.0, 0, 1);
22 }
23 .button:hover {
24     -fx-background-color: linear-gradient(#2A5058, #61a2b1);
25 }
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.geometry.Insets?>
4 <?import javafx.scene.control.*?>
5 <?import javafx.scene.layout.*?>
6 <?import javafx.scene.text.Text?>
7
8 <GridPane xmlns="http://javafx.com/javafx/8" xmlns:fx="http://javafx.com/fxml/1"
9         fx:controller="controller.FirstController" stylesheets="@fxml_example.css"
10        alignment="center" hgap="10" vgap="10">
11     <padding>
12         <Insets top="25" right="25" bottom="10" left="25"/>
13     </padding>
14     <Text text="Welcome" id="welcome-text"
15         GridPane.columnIndex="0" GridPane.rowIndex="0" GridPane.columnSpan="2"/>
16     <Label text="User Name:"
17         GridPane.columnIndex="0" GridPane.rowIndex="1"/>
18     <TextField
19         GridPane.columnIndex="1" GridPane.rowIndex="1"/>
20     <Label text="Password:"
21         GridPane.columnIndex="0" GridPane.rowIndex="2"/>
22     <PasswordField fx:id="passwordField"
23         GridPane.columnIndex="1" GridPane.rowIndex="2"/>
24     <HBox spacing="10" alignment="bottom_right"
25         GridPane.columnIndex="1" GridPane.rowIndex="4">
26         <Button text="Sign In" onAction="#handleButtonAction"/>
27     </HBox>
28     <Text fx:id="actiontarget"
29         GridPane.columnIndex="0" GridPane.columnSpan="2"
30         GridPane.halignment="RIGHT" GridPane.rowIndex="6"/>
31 </GridPane>

```

Résultat avec/sans CSS



- `Stage` est le container de plus haut niveau.
- Chaque page a son `Stage`.
- Le `Stage` est rempli avec une `Scene`.
- `Scene` est le container pour tout le contenu du graphe de l'application.
- L'application doit définir le nœud racine de la scène.
- La taille de la scène peut être définie par le programme pendant la construction.
- Sinon, les dimensions préférées sont utilisées.
- La scène est remplie avec les nœuds fils, généralement des panneaux (`Pane`).

Stage et Scene

```
1  public class HelloWorld extends Application {  
2  
3      @Override  
4      public void start(Stage stage) {  
5          Text text = new Text(10, 40, "Hello World!");  
6          text.setFont(new Font(40));  
7          Scene scene = new Scene(new Group(text));  
8  
9          stage.setTitle("Welcome to JavaFX!");  
10         stage.setScene(scene);  
11         stage.sizeToScene();  
12         stage.show();  
13     }  
14  
15     public static void main(String[] args) {  
16         Application.launch(args);  
17     }  
18 }
```

Panneaux pour l'affichage

`javafx.scene.layout.Pane`

- `BorderPane` : les fils sont disposés en 5 positions : en haut, à droite, en bas, à gauche et au centre.
- `AnchorPane` : les emplacements des enfants sont indiquées relativement aux bords.
- `TabPane` : pour l'affichage avec onglets.
- `GridPane` : pour l'affichage sous forme de tableau flexible.
- `SplitPane` : pour l'affichage de plusieurs parties.
- `ScrollPane` : pour l'affichage déroulant.
- `StackPane` : pour l'affichage sous forme de pile.
- `HBox` : pour l'affichage des enfants alignés horizontalement.
- `VBox` : pour l'affichage des enfants alignés verticalement.

Structures de contrôle

`javafx.scene.control.Control`

- `ChoiceBox` : boîte de choix prédéfinis.
- `HTML editor` : éditeur HTML
- `Label` : texte non éditable.
- `ListView` : une liste horizontale ou verticale.
- `MenuBar` : le menu en haut de la fenêtre.
- `Spinner` : liste déroulante.
- `TableView` : un tableau à deux dimensions.
- `ToolBar` : barre d'affichage, généralement de boutons.
- `TreeView` : une arborescence.

Structures de contrôle

`javafx.scene.control.Control`

- `Button` : bouton (texte ou graphique).
- `CheckBox` : case à cocher.
- `HyperLink` : lien hypertexte.
- `MenuButton` : ouverture d'un menu contextuel.
- `ToggleButton` : case à choisir (dans un `ToggleGroup`).
- `RadioButton` : même chose, mais avec au moins un sélectionné.
- `ColorPicker` : choix de couleur.
- `ComboBox` : liste de choix.
- `DatePicker` : choix de date.
- `TextArea` : texte multi-line.
- `TextField` : texte d'une ligne (éventuellement avec prompt).

- `ObservableList<E>` : liste permettant les écouteurs de suivre les changement.
- `ObservableMap<K, V>` : table de hachage permettant les écouteurs de suivre les changement.
- `FXCollections` : classe utilitaire composée de méthodes statiques qui font la même chose que les méthodes de la classe `java.util.Collections`, par exemple :
 - `observableArrayList()`
 - `observableArrayList(E... items)`
 - `observableHashMap()`
 - ...

TableView

```
1 public class Person {
2     private String firstName, lastName, phone;
3     public String getFirstName() {
4         return firstName;
5     }
6     public void setFirstName(String value) {
7         firstName = value;
8     }
9     public String getLastName() {
10        return lastName;
11    }
12    public void setLastName(String value) {
13        lastName = value;
14    }
15    public String getPhone() {
16        return phone;
17    }
18    public void setPhone(String value) {
19        phone = value;
20    }
21 }
```

Dans la vue :

```
1 <TableView fx:id="table">
2   <columns>
3     <TableColumn fx:id="firstNameColumn"
4       text="First Name" prefWidth="180.0"/>
5     <TableColumn fx:id="lastNameColumn"
6       text="Last Name" prefWidth="180.0"/>
7     <TableColumn fx:id="phoneColumn"
8       text="Phone Number" prefWidth="180.0"/>
9   </columns>
10 </TableView>
```

Dans le contrôleur :

```
1  @FXML
2  private TableView<Person> table;
3  @FXML
4  private TableColumn<Person, String> firstNameColumn;
5  @FXML
6  private TableColumn<Person, String> lastNameColumn;
7  @FXML
8  private TableColumn<Person, String> phoneColumn;
9  ...
10     ObservableList<Person> members =
11         model.getTeamMembers()
12     table.setItems(members);
13     firstNameColumn.setCellValueFactory(
14         new PropertyValueFactory<>("firstName"));
15     lastNameColumn.setCellValueFactory(
16         new PropertyValueFactory<>("lastName"));
17     phoneColumn.setCellValueFactory(
18         new PropertyValueFactory<>("phone"));
```

TableView

First Name	Last Name	Phone Number
FirstName1	LastName1	Phone1
FirstName2	LastName2	Phone2
FirstName3	LastName3	Phone3
FirstName4	LastName4	Phone4
FirstName5	LastName5	Phone5
FirstName6	LastName6	Phone6
FirstName7	LastName7	Phone7
FirstName8	LastName8	Phone8
FirstName9	LastName9	Phone9
FirstName10	LastName10	Phone10
FirstName11	LastName11	Phone11
FirstName12	LastName12	Phone12

- `FadeTransition` : apparition/disparition graduelle.
- `FillTransition` : changement de la couleur d'objet.
- `PathTransition` : déplacement d'objet le long d'une trajectoire.
- `PauseTransition` : exécution à la fin d'un temps prédéfini.
- `RotateTransition` : rotation d'objet.
- `ScaleTransition` : agrandissement/diminution d'objet.
- `TranslateTransition` : déplacement d'objet en ligne droite.

Animations `javafx.animation`

```
1  import javafx.scene.shape.*;
2  import javafx.animation.transition.*;
3  ...
4      Rectangle rect = new Rectangle (100, 40, 100, 100);
5      rect.setArcHeight(50);
6      rect.setArcWidth(50);
7      rect.setFill(Color.VIOLET);
8
9      ScaleTransition st =
10          new ScaleTransition(Duration.millis(2000), rect);
11      st.setByX(1.5f);
12      st.setByY(1.5f);
13      st.setCycleCount(4f);
14      st.setAutoReverse(true);
15
16      st.play();
17  ...
```


Propriétés `javafx.beans.property`

```
1 public class Person {
2     private StringProperty firstName;
3
4     public Person(String name) {
5         firstName = new SimpleStringProperty(name);
6     }
7
8     public void final setFirstName(String value) {
9         firstNameProperty().set(value);
10    }
11
12    public final String getFirstName() {
13        return firstNameProperty().get();
14    }
15
16    public StringProperty firstNameProperty() {
17        return firstName;
18    }
```

```
1 public class Controller {  
2     @FXML  
3     private Label firstNameLabel;  
4     private Person person;  
5  
6     public void initialize() {  
7         firstNameLabel.textProperty()  
8             .bind(person.firstNameProperty());  
9     }  
10 }
```

On peut faire des opérations sur les liaisons.

Surface d'un triangle

$(x1*y2 + x2*y3 + x3*y1 - x1*y3 - x2*y1 - x3*y2) / 2.$

```
1 IntegerProperty x1 = new SimpleIntegerProperty(0);
2 ...
3 NumberBinding area = x1.multiply(y2)
4                       .add(x2.multiply(y3))
5                       .add(x3.multiply(y1))
6                       .subtract(x1.multiply(y3))
7                       .subtract(x2.multiply(y1))
8                       .subtract(x3.multiply(y2))
9                       .divide(2.0D);
```

Si $a + b > c, b + c > a, c + a > b$

alors $s = \sqrt{s * (s - a) * (s - b) * (s - c)}$ où $s = (a + b + c)/2$

```
1 DoubleProperty a = new SimpleDoubleProperty(0);
2 DoubleProperty b = new SimpleDoubleProperty(0);
3 DoubleProperty c = new SimpleDoubleProperty(0);
4 DoubleBinding s = a.add(b).add(c).divide(2.0D);
5 DoubleBinding areaSquared =
6     new When(a.add(b).greaterThan(c)
7         .and(b.add(c).greaterThan(a))
8         .and(c.add(a).greaterThan(b)))
9     .then(s.multiply(s.subtract(a))
10         .multiply(s.subtract(b))
11         .multiply(s.subtract(c)))
12     .otherwise(0.0D);
```

- Il faut récupérer l'archive JavaFx sur Teams et désarchiver.
- Il faut ajouter `-module-path /Path/To/JavaFX/lib`
`-add-modules=javafx.controls,javafx.fxml`
- Sous Eclipse, il faut décocher *"Use the -XstartOnFirstThread argument when launching with SWT"*